Computer Networks Lab Assessment 6

Student: Upayan Mazumder
Registration number: 24BDS0367

TCP Server–Client Socket Program

The server echoes messages from a local TCP client. Both programs print the received data.

Compile and run

```
cc -std=c11 -Wall -Wextra -o server server.c
cc -std=c11 -Wall -Wextra -o client client.c
./server                    # first terminal
./client "Hello, TCP!"      # second terminal
```

Current program output

The server listens on 127.0.0.1:8080.

```
$ /tmp/assessment6-tcp-server-8080
Listening on 127.0.0.1:8080
Client: Hello, TCP!
Client: Second message
```

```
$ /tmp/assessment6-tcp-client-8080 "Hello, TCP!"
Hello, TCP!
$ /tmp/assessment6-tcp-client-8080 "Second message"
Second message
```

Original terminal screenshot

This earlier run used the server version without message printing.

```
bash-5.3$ gcc -std=c11 -Wall -Wextra -o server server.c
bash-5.3$ ./server
Listening on 127.0.0.1:8080
```

```
bash-5.3$ cc -std=c11 -Wall -Wextra -o client client.c
bash-5.3$ ./client "Hello, TCP!"
Hello, TCP!
bash-5.3$
bash-5.3$
bash-5.3$ ./client "Hello, TCP!"
Hello, TCP!
bash-5.3$
```

server.c

```c
#include <arpa/inet.h>
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <sys/socket.h>
#include <unistd.h>

#ifndef PORT
#define PORT 8080
#endif

static int send_all(int fd, const char *data, size_t length)
{
    size_t sent = 0;
    while (sent < length) {
        ssize_t count = send(fd, data + sent, length - sent, 0);
        if (count < 0 && errno == EINTR)
            continue;
        if (count <= 0)
            return -1;
        sent += (size_t)count;
    }
    return 0;
}

int main(void)
{
    signal(SIGPIPE, SIG_IGN);

    int listener = socket(AF_INET, SOCK_STREAM, 0);
    if (listener < 0) {
        perror("socket");
        return 1;
    }

    int reuse = 1;
    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse)) < 0) {
        perror("setsockopt");
        close(listener);
        return 1;
    }

    struct sockaddr_in address = {
        .sin_family = AF_INET,
        .sin_port = htons(PORT),
        .sin_addr.s_addr = htonl(INADDR_LOOPBACK)
    };
    if (bind(listener, (struct sockaddr *)&address, sizeof(address)) < 0 ||
```

server.c

```
49          listen(listener, 5) < 0) {
50          perror("bind/listen");
51          close(listener);
52          return 1;
53      }
54
55      printf("Listening on 127.0.0.1:%d\n", PORT);
56      fflush(stdout);
57
58      for (;;) {
59          int client = accept(listener, NULL, NULL);
60          if (client < 0) {
61              if (errno == EINTR)
62                  continue;
63              perror("accept");
64              close(listener);
65              return 1;
66          }
67
68          char buffer[4096];
69          int logged = 0;
70          for (;;) {
71              ssize_t count = recv(client, buffer, sizeof(buffer), 0);
72              if (count < 0 && errno == EINTR)
73                  continue;
74              if (count < 0) {
75                  perror("recv");
76                  break;
77              }
78              if (count == 0)
79                  break;
80              if (!logged) {
81                  fputs("Client: ", stdout);
82                  logged = 1;
83              }
84              fwrite(buffer, 1, (size_t)count, stdout);
85              if (send_all(client, buffer, (size_t)count) < 0) {
86                  perror("send");
87                  break;
88              }
89          }
90          if (logged) {
91              putchar('\n');
92              fflush(stdout);
93          }
94          close(client);
95      }
96  }
```

client.c

```c
#include <arpa/inet.h>
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#ifndef PORT
#define PORT 8080
#endif

static int send_all(int fd, const char *data, size_t length)
{
    size_t sent = 0;
    while (sent < length) {
        ssize_t count = send(fd, data + sent, length - sent, 0);
        if (count < 0 && errno == EINTR)
            continue;
        if (count <= 0)
            return -1;
        sent += (size_t)count;
    }
    return 0;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s \"message\"\n", argv[0]);
        return 1;
    }
    signal(SIGPIPE, SIG_IGN);

    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        return 1;
    }

    struct sockaddr_in address = {
        .sin_family = AF_INET,
```

client.c

```
43          .sin_port = htons(PORT),
44          .sin_addr.s_addr = htonl(INADDR_LOOPBACK)
45      };
46      if (connect(fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
47          perror("connect");
48          close(fd);
49          return 1;
50      }
51
52      if (send_all(fd, argv[1], strlen(argv[1])) < 0 ||
53          shutdown(fd, SHUT_WR) < 0) {
54          perror("send/shutdown");
55          close(fd);
56          return 1;
57      }
58
59      char buffer[4096];
60      for (;;) {
61          ssize_t count = recv(fd, buffer, sizeof(buffer), 0);
62          if (count < 0 && errno == EINTR)
63              continue;
64          if (count < 0) {
65              perror("recv");
66              close(fd);
67              return 1;
68          }
69          if (count == 0)
70              break;
71          if (fwrite(buffer, 1, (size_t)count, stdout) != (size_t)count) {
72              perror("stdout");
73              close(fd);
74              return 1;
75          }
76      }
77      close(fd);
78      if (putchar('\n') == EOF || fflush(stdout) == EOF) {
79          perror("stdout");
80          return 1;
81      }
82      return 0;
83  }
```

server.c

```
#include <arpa/inet.h>
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <sys/socket.h>
#include <unistd.h>

#ifndef PORT
#define PORT 8080
#endif

static int send_all(int fd, const char *data, size_t length)
{
    size_t sent = 0;
    while (sent < length) {
        ssize_t count = send(fd, data + sent, length - sent, 0);
        if (count < 0 && errno == EINTR)
            continue;
        if (count <= 0)
            return -1;
        sent += (size_t)count;
    }
    return 0;
}

int main(void)
{
    signal(SIGPIPE, SIG_IGN);

    int listener = socket(AF_INET, SOCK_STREAM, 0);
    if (listener < 0) {
        perror("socket");
        return 1;
    }

    int reuse = 1;
    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse)) < 0) {
        perror("setsockopt");
        close(listener);
        return 1;
    }

    struct sockaddr_in address = {
        .sin_family = AF_INET,
        .sin_port = htons(PORT),
        .sin_addr.s_addr = htonl(INADDR_LOOPBACK)
    };
    if (bind(listener, (struct sockaddr *)&address, sizeof(address)) < 0 ||
```

server.c

```
        listen(listener, 5) < 0) {
        perror("bind/listen");
        close(listener);
        return 1;
    }

    printf("Listening on 127.0.0.1:%d\n", PORT);
    fflush(stdout);

    for (;;) {
        int client = accept(listener, NULL, NULL);
        if (client < 0) {
            if (errno == EINTR)
                continue;
            perror("accept");
            close(listener);
            return 1;
        }

        char buffer[4096];
        int logged = 0;
        for (;;) {
            ssize_t count = recv(client, buffer, sizeof(buffer), 0);
            if (count < 0 && errno == EINTR)
                continue;
            if (count < 0) {
                perror("recv");
                break;
            }
            if (count == 0)
                break;
            if (!logged) {
                fputs("Client: ", stdout);
                logged = 1;
            }
            fwrite(buffer, 1, (size_t)count, stdout);
            if (send_all(client, buffer, (size_t)count) < 0) {
                perror("send");
                break;
            }
        }
        if (logged) {
            putchar('\n');
            fflush(stdout);
        }
        close(client);
    }
}
```

client.c

```
#include <arpa/inet.h>
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#ifndef PORT
#define PORT 8080
#endif

static int send_all(int fd, const char *data, size_t length)
{
    size_t sent = 0;
    while (sent < length) {
        ssize_t count = send(fd, data + sent, length - sent, 0);
        if (count < 0 && errno == EINTR)
            continue;
        if (count <= 0)
            return -1;
        sent += (size_t)count;
    }
    return 0;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s \"message\"\n", argv[0]);
        return 1;
    }
    signal(SIGPIPE, SIG_IGN);

    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        return 1;
    }

    struct sockaddr_in address = {
        .sin_family = AF_INET,
```

client.c

```
        .sin_port = htons(PORT),
        .sin_addr.s_addr = htonl(INADDR_LOOPBACK)
    };
    if (connect(fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("connect");
        close(fd);
        return 1;
    }

    if (send_all(fd, argv[1], strlen(argv[1])) < 0 ||
        shutdown(fd, SHUT_WR) < 0) {
        perror("send/shutdown");
        close(fd);
        return 1;
    }

    char buffer[4096];
    for (;;) {
        ssize_t count = recv(fd, buffer, sizeof(buffer), 0);
        if (count < 0 && errno == EINTR)
            continue;
        if (count < 0) {
            perror("recv");
            close(fd);
            return 1;
        }
        if (count == 0)
            break;
        if (fwrite(buffer, 1, (size_t)count, stdout) != (size_t)count) {
            perror("stdout");
            close(fd);
            return 1;
        }
    }
    close(fd);
    if (putchar('\n') == EOF || fflush(stdout) == EOF) {
        perror("stdout");
        return 1;
    }
    return 0;
}
```